



Căutare binară

Varianta clasică · Variații · STL · Complexitate

Căutare binară pe rezultat

Andrei Mihai Ciobanu

Algolymp Summer School

Căutarea binară a unui element

Variații

Variații în STL

De ce $O(\log n)$?

Căutare binară pe rezultat

Recapitulare

Resurse

Probleme de antrenament

Căutarea binară a unui element

Motivație

Problemă: avem un șir *sortat* cu n elemente și vrem să știm dacă valoarea x se află în el.

Căutare secvențială: parcurgem elementele pe rând $\rightarrow$ în cel mai rău caz, n comparații.

Căutare binară: folosim faptul că șirul e sortat și eliminăm jumătate din elemente la fiecare pas.

n	Secvențial	Binar
10	10 pași	4 pași
100	100 pași	7 pași
1 000 000	10^6 pași	20 pași

Condiții

- Șirul trebuie să fie **sortat** (crescător sau descrescător)

Păstrăm un interval $[st, dr]$ care conține răspunsul. La fiecare pas:

1. Calculăm mijlocul: $mij = (st + dr) / 2$
2. Comparăm $v[mij]$ cu x :
 - $v[mij] = x \rightarrow$ **găsit!**
 - $v[mij] < x \rightarrow st = mij + 1$ — căutăm în dreapta
 - $v[mij] > x \rightarrow dr = mij - 1$ — căutăm în stânga

Ne oprim când găsim valoarea sau când intervalul devine gol ($st > dr$).

Algoritm pas cu pas

Șir sortat: $v = [2, 5, 8, 11, 14, 17, 20, 23, 26, 29, 32]$, căutăm $x = 23$.

- $st = 0, dr = 10 \Rightarrow mij = 5, v[5] = 17 < 23 \Rightarrow st = 6$
- $st = 6, dr = 10 \Rightarrow mij = 8, v[8] = 26 > 23 \Rightarrow dr = 7$
- $st = 6, dr = 7 \Rightarrow mij = 6, v[6] = 20 < 23 \Rightarrow st = 7$
- $st = 7, dr = 7 \Rightarrow mij = 7, v[7] = 23 = 23 \Rightarrow$ **găsit la poziția 7 ✓**

Vizualizare — intervalul se înjumătățește

	0	1	2	3	4	5	6	7	8	9	10
pas 1: $mij = 5$	2	5	8	11	14	17	20	23	26	29	32
pas 2: $mij = 8$	2	5	8	11	14	17	20	23	26	29	32
pas 3: $mij = 6$	2	5	8	11	14	17	20	23	26	29	32
pas 4: gasit!	2	5	8	11	14	17	20	23	26	29	32

La fiecare pas, intervalul $[st, dr]$ se înjumătățește.

Căutare secvențială — implementare

```
1  int n, x;
2  cin >> n >> x;
3  vector<int> v(n);
4  for (int i = 0; i < n; i++) {
5      cin >> v[i];
6  }
7
8  int poz = -1;
9  for (int i = 0; i < n; i++) {
10     if (v[i] == x) {
11         poz = i;
12         break;
13     }
14 }
15
16 cout << poz;
```

- Complexitate: $O(n)$ — în cel mai rău caz parcurgem tot șirul
- Funcționează pe orice șir, **sortat sau nu**

Căutare binară — implementare

```
1  int n, x;
2  cin >> n >> x;
3  vector<int> v(n);
4  for (int i = 0; i < n; i++) {
5      cin >> v[i];
6  }
7
8  int st = 0, dr = n - 1, poz = -1;
9  while (st <= dr) {
10     int mij = (st + dr) / 2;
11     if (v[mij] == x) {
12         poz = mij;
13         break;
14     } else if (v[mij] < x) {
15         st = mij + 1;  // cautam in dreapta
16     } else {
17         dr = mij - 1;  // cautam in stanga
18     }
19 }
20
21 cout << poz;
```

- Complexitate: $O(\log n)$ — șirul trebuie să fie sortat

Variații

Lower bound — upper bound

Căutarea clasică găsește o **aparitie oarecare** a lui x . Adesea avem nevoie de:

- **Lower bound:** prima poziție cu $v[poz] \geq x$
- **Upper bound:** prima poziție cu $v[poz] > x$

Exemplu: $v = [1, 2, 2, 2, 3, 5]$, $x = 2$

	Poziție	Valoare
Lower bound (≥ 2)	1	$v[1] = 2$
Upper bound (> 2)	4	$v[4] = 3$

Numărul de apariții ale lui x :

$$\text{upper_bound} - \text{lower_bound} = 4 - 1 = 3$$

Implementare lower_bound

```
1 // prima pozitie cu v[poz] >= x
2 // rez = n daca toate elementele < x
3 int st = 0, dr = n - 1, rez = n;
4 while (st <= dr) {
5     int mij = (st + dr) / 2;
6     if (v[mij] >= x) {
7         rez = mij;    // candidat, continuam in stanga
8         dr = mij - 1;
9     } else {
10         st = mij + 1;
11     }
12 }
```

Când $v[mij] \geq x$, **salvăm** poziția și căutăm mai la stânga un candidat mai bun.

Implementare upper_bound

```
1 // prima pozitie cu v[poz] > x
2 // rez = n daca toate elementele <= x
3 int st = 0, dr = n - 1, rez = n;
4 while (st <= dr) {
5     int mij = (st + dr) / 2;
6     if (v[mij] > x) {
7         rez = mij;    // candidat, continuam in stanga
8         dr = mij - 1;
9     } else {
10         st = mij + 1;
11     }
12 }
```

Singura diferență față de lower_bound: condiția $v[mij] > x$ în loc de $v[mij] \geq x$.

Ultima poziție cu $v[poz] < x / \leq x$

Aceleași idei ne dau și pozițiile din partea cealaltă:

- Ultima poziție cu $v[poz] < x = \text{lower_bound}(x) - 1$
- Ultima poziție cu $v[poz] \leq x = \text{upper_bound}(x) - 1$

Sau adaptăm direct implementarea, inversând sensul căutării:

```
1 // ultima pozitie cu v[poz] <= x
2 // rez = -1 daca toate elementele > x
3 int st = 0, dr = n - 1, rez = -1;
4 while (st <= dr) {
5     int mij = (st + dr) / 2;
6     if (v[mij] <= x) {
7         rez = mij;      // candidat, continuam in dreapta
8         st = mij + 1;
9     } else {
10         dr = mij - 1;
11     }
12 }
```

Pentru $< x$: schimbăm condiția în $v[mij] < x$.

Variații în STL

STL: binary_search, lower_bound, upper_bound

C++ oferă aceste funcții gata implementate în `<algorithm>`, toate $O(\log n)$:

```
1 #include <algorithm>
2 vector<int> v = {1, 2, 2, 2, 3, 5};
3
4 // Exista x in v? (true/false)
5 bool exista = binary_search(v.begin(), v.end(), 2);
6
7 // Iteratori
8 auto it1 = lower_bound(v.begin(), v.end(), 2);
9 auto it2 = upper_bound(v.begin(), v.end(), 2);
10
11 // Pozitii (indici)
12 int poz1 = it1 - v.begin(); // poz1 = 1 (prima aparitie)
13 int poz2 = it2 - v.begin(); // poz2 = 4 (dupa ultima)
14
15 // Numarul de aparitii ale lui x
16 int cate = it2 - it1;      // cate = 3
```

▷ vector, iterator — din STL (vezi slide-ul Resurse)

STL — numărare pe interval $[x, y]$

```
1  vector<int> v = {23, 2, 5, 26, 11, 17, 14, 8, 20};
2  // sirul trebuie sortat inainte!
3  sort(v.begin(), v.end());
4
5  int x = 8, y = 20;
6
7  // prima pozitie cu v[poz] >= x
8  int st = lower_bound(v.begin(), v.end(), x) - v.begin();
9
10 // prima pozitie cu v[poz] > y
11 int dr = upper_bound(v.begin(), v.end(), y) - v.begin();
12
13 int cate = dr - st; // cate = 5 (8, 11, 14, 17, 20)
```

- Idee frecventă: interogări pe şir sortat în $O(\log n)$ în loc de $O(n)$

De ce $O(\log n)$?

De ce $O(\log n)$?

La fiecare pas, intervalul $[st, dr]$ se **înjumătățește**:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow 1$$

Numărul de pași până ajungem la 1: $\log_2 n$.

n	$\log_2 n$	Pași maximi
10	3.32	4
1 000	9.97	10
10^6	19.93	20
10^9	29.90	30
10^{18}	59.79	60

Chiar și pentru $n = 10^{18}$, avem cel mult 60 de pași!

Căutare binară pe rezultat

Căutare binară pe rezultat — exemplu

Cea mai puternică aplicație: nu căutăm o valoare într-un șir, ci **cel mai bun răspuns posibil**.

Problemă

Avem n bețe de lungimi $d_1, d_2, \dots, d_n$. Fiecare băț poate fi tăiat în bucăți mai mici de lungimi întregi. Determinați **lungimea maximă** L pentru care obținem în total **cel puțin** A bucăți.

Restricții: $1 \leq n \leq 10^5$, $1 \leq d_i \leq 10^9$, $1 \leq A \leq 10^9$.

Exemplu: $d = [7, 4, 9, 5]$, $A = 6 \Rightarrow$ răspuns $L = 3$ (obținem $2 + 1 + 3 + 1 = 7 \geq 6$ bucăți).

Căutare binară pe rezultat — soluție naivă

Cea mai simplă idee: încercăm pe rând fiecare lungime posibilă, de la cea mai mare la cea mai mică, și ne oprim la prima care funcționează.

```
1 long long taieBucati(vector<int>& d, int L) {
2     long long bucati = 0;
3     for (int i = 0; i < d.size(); ++i) {
4         bucati += d[i] / L;
5     }
6     return bucati;
7 }
8
9 for (int L = maxim; L >= 1; L--) {
10     if (taieBucati(d, L) >= A) {
11         cout << L;
12         break;
13     }
14 }
```

- Complexitate: $O(n \cdot maxim)$, unde *maxim* este lungimea maximă a unui băț
- Pentru $maxim = 10^9$ și $n = 10^5$: mult prea lent

Căutare binară pe rezultat — observație

Pe măsură ce L crește, numărul de bucăți obținute de $\text{taieBucati}(d, L)$ **scade** (sau rămâne la fel).

Exemplu: $d = [7, 4, 9, 5]$, verificăm $\text{taieBucati}(L) \geq A$, $A = 6$

L	1	2	3	4	5	6	7	8	9
$\text{taieBucati}(L)$	25	11	7	5	3	2	2	1	1
$\geq A?$	✓	✓	✓	×	×	×	×	×	×

Observație

Șirul de valori $\text{taieBucati}(L)$ e **monoton descrescător**: odată ce scade sub A , rămâne sub A . Îl putem privi ca pe un șir sortat descrescător (la fel ca la *lower_bound*, vezi secțiunea *Variații*) și **eliminăm jumătate din candidați** la fiecare iterație.

Căutare binară pe rezultat — soluție optimizată

```
1 long long taieBucati(vector<int>& d, int L) {
2     long long bucati = 0;
3     for (int i = 0; i < d.size(); ++i) {
4         bucati += d[i] / L;
5     }
6     return bucati;
7 }
8
9 int st = 1, dr = maxim, rez = 0;
10 while (st <= dr) {
11     int mij = (st + dr) / 2;
12     if (taieBucati(d, mij) >= A) {
13         rez = mij;      // functioneaza, incercam mai mare
14         st = mij + 1;
15     } else {
16         dr = mij - 1;   // nu functioneaza, incercam mai mic
17     }
18 }
19 cout << rez;
```

- Complexitate: $O(n \log(maxim))$ — pentru aceleași date,
 $\approx 10^5 \cdot 30 = 3 \times 10^6$

Căutare binară pe rezultat — observație: overflow

Observație: overflow

La căutarea binară pe rezultat, intervalul $[st, dr]$ poate fi foarte mare (de exemplu $dr = maxim \approx 10^9$), deci $st + dr$ riscă să depășească un `int`.

În astfel de cazuri, ori folosim `long long` pentru st , dr , mij , ori schimbăm formula pentru mijloc: $mij = st + (dr - st) / 2$.

Comparativ, la căutarea binară a unui element, $[st, dr]$ e mărginit de n (indici de tablou), deci acest risc de overflow nu apare de obicei.

Căutare binară pe rezultat — condiția de aplicabilitate

Căutarea binară pe rezultat funcționează **doar** când există o structură de tipul:

... ✓ ✓ ✓ × × × × ...

adică există un **singur prag** unde condiția $taieBucati(L) \geq A$ trece din adevărat în fals (sau invers) — exact ca în tabelul de la exemplul cu bețele.

Verificare practică

Înainte să aplici căutarea binară pe rezultat: întreabă-te “*pentru orice L care merge, merge și $L - 1$?*” Dacă da pentru toate valorile, funcția e monotonă și poți aplica căutarea binară.

Exemple care nu respectă condiția

- Șir nesortat $\Rightarrow$ nu poți aplica căutarea binară pe element
- Funcție de tip $\checkmark \times \checkmark \times \checkmark \Rightarrow$ nu e monotonă

Recapitulare

Recapitulare

Variantă	Timp	Spațiu
Căutare secvențială	$O(n)$	$O(1)$
Căutare binară (element)	$O(\log n)$	$O(1)$
STL <code>lower/upper_bound</code>	$O(\log n)$	$O(1)$
Căutare binară pe rezultat	$O(\log n \cdot f(n))$	depinde

- Șirul trebuie sortat înainte de căutare binară
- Pentru intervale $[st, dr]$ foarte mari, folosim `long long` sau formula sigură $mij = st + (dr - st) / 2$
- `lower_bound` $\rightarrow$ prima poz. cu $\geq x$; `upper_bound` $\rightarrow$ prima poz. cu $> x$

Resurse

Română

- **pbinfo.ro** — Căutarea binară (RO)
- **infoas.ro** — Căutare binară în C++ (RO)

Engleză

- **cp-algorithms.com** — Binary Search (EN, detaliat, include și căutarea binară pe numere reale)
- **usaco.guide** — Binary Search on Monotonic Functions (EN)

Documentație C++

- **cppreference.com** — `lower_bound` / `upper_bound`

Fun fact

- **cp-algorithms.com** — Ternary Search (EN) — căutare **ternară**, folosită pentru funcții unimodale (ex. $ax^2 + bx + c$)

Probleme de antrenament

Probleme de antrenament

Ușoare

- **Classic Binary Search #3963** (AlgoCode)
- **First Position Not Less Than Query #4202** (AlgoCode)

Medii

- **Search Task #1009** (AlgoCode)
- **Snack Route #3962** (AlgoCode)

Grea

- **Search Task #1008** (AlgoCode)

Olimpiadă

- **Buldo** (OJI 2020, clasa IX)
- **Macarie** (OJI 2024, clasa IX)